

deviates first exceeds λ , their number (less 1) is a Poisson deviate k . Also, as explained for the gamma distribution, we can multiply uniform deviates from $U(0, 1)$ instead of adding deviates from Exponential (1).

These ideas produce the following routine.

```

struct Poissondev : Ran {
Structure for Poisson deviates.
    Doub lambda, sqlam, loglam, lamexp, lambold;
    VecDoub logfact;
    Int swch;
    Poissondev(Doub llambda, Ullong i) : Ran(i), lambda(llambda),
        logfact(1024,-1.), lambold(-1.) {}
Constructor arguments are  $\lambda$  and a random sequence seed.
    Int dev() {
Return a Poisson deviate using the most recently set value of  $\lambda$ .
        Doub u,u2,v,v2,p,t,lfac;
        Int k;
        if (lambda < 5.) {
            Will use product of uniforms method.
            if (lambda != lambold) lamexp=exp(-lambda);
            k = -1;
            t=1.;
            do {
                ++k;
                t *= doub();
            } while (t > lamexp);
        } else {
            Will use ratio-of-uniforms method.
            if (lambda != lambold) {
                sqlam = sqrt(lambda);
                loglam = log(lambda);
            }
            for (;;) {
                u = 0.64*doub();
                v = -0.68 + 1.28*doub();
                if (lambda > 13.5) {
                    Outer squeeze for fast rejection.
                    v2 = SQR(v);
                    if (v >= 0.) {if (v2 > 6.5*u*(0.64-u)*(u+0.2)) continue;}
                    else {if (v2 > 9.6*u*(0.66-u)*(u+0.07)) continue;}
                }
                k = Int(floor(sqlam*(v/u)+lambda+0.5));
                if (k < 0) continue;
                u2 = SQR(u);
                if (lambda > 13.5) {
                    Inner squeeze for fast acceptance.
                    if (v >= 0.) {if (v2 < 15.2*u2*(0.61-u)*(0.8-u)) break;}
                    else {if (v2 < 6.76*u2*(0.62-u)*(1.4-u)) break;}
                }
                if (k < 1024) {
                    if (logfact[k] < 0.) logfact[k] = gammln(k+1.);
                    lfac = logfact[k];
                } else lfac = gammln(k+1.);
                p = sqlam*exp(-lambda + k*loglam - lfac);
                if (u2 < p) break;
            }
        }
        lambold = lambda;
        return k;
    }
    Int dev(Doub llambda) {
Reset  $\lambda$  and then return a Poisson deviate.
        lambda = llambda;
        return dev();
    }
};

```

deviates.h

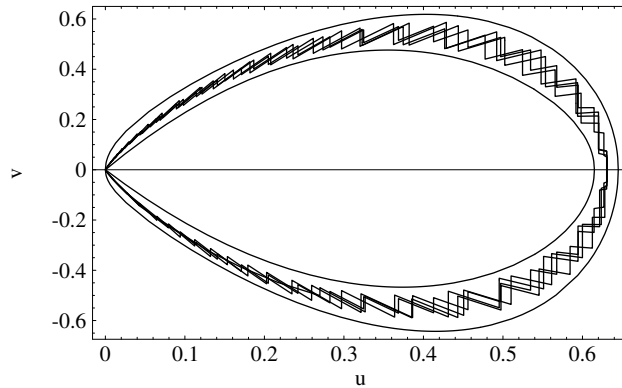


Figure 7.3.5. Ratio-of-uniforms method as applied to the generation of binomial deviates. Points are chosen randomly in the (u, v) -plane. The smooth curves are inner and outer squeezes. The jagged curves correspond to various binomial distributions with $n > 64$ and $np > 30$. An evaluation of the binomial probability is required only when the random point falls between the smooth curves.

In the regime $\lambda > 13.5$, the above code uses about 3.3 uniform deviates per output Poisson deviate and does 0.4 evaluations of the exact probability (costing an exponential and, for large k , a call to `gammaln`).

`Poissondev` is slightly faster if you draw many deviates with the same value λ , using the `dev` function with no arguments, than if you vary λ on each call, using the one-argument overloaded form of `dev` (which is provided for just that purpose). The difference is just an extra exponential ($\lambda < 5$) or square root and logarithm ($\lambda \geq 5$). Note also the object's table of previously computed log-factorials. If your λ 's are as large as $\sim 10^3$, you might want to make the table larger.

7.3.13 Binomial Deviates

The generation of binomial deviates $k \sim \text{Binomial}(n, p)$ involves many of the same ideas as for Poisson deviates. The distribution is again integer-valued, so we use the same trick to convert it into a stepped continuous distribution. We can always restrict attention to the case $p \leq 0.5$, since the distribution's symmetries let us trivially recover the case $p > 0.5$.

When $n > 64$ and $np > 30$, we use the ratio-of-uniforms method, with squeezes shown in Figure 7.3.5. The cost is about 3.2 uniform deviates, plus 0.4 evaluations of the exact probability, per binomial deviate.

It would be foolish to waste much thought on the case where $n > 64$ and $np < 30$, because it is so easy simply to tabulate the cdf, say for $0 \leq k < 64$, and then loop over k 's until the right one is found. (A bisection search, implemented below, is even better.) With a cdf table of length 64, the neglected probability at the end of the table is never larger than $\sim 10^{-20}$. (At 10^9 deviates per second, you could run 3000 years before losing a deviate.)

What is left is the interesting case $n < 64$, which we will explore in some detail, because it demonstrates the important concept of *bit-parallel random comparison*.

Analogous to the methods for gamma deviates with small integer a and for Poisson deviates with small λ , is this direct method for binomial deviates: Generate n uniform deviates in $U(0, 1)$. Count the number of them $< p$. Return the count as

$k \sim \text{Binomial}(n, p)$. Indeed this is essentially the definition of a binomial process!

The problem with the direct method is that it seems to require n uniform deviates, even when the mean value of k is much smaller. Would you be surprised if we told you that for $n \leq 64$ you can achieve the same goal with at most *seven* 64-bit uniform deviates, on average? Here is how.

Expand $p < 1$ into its first 5 bits, plus a residual,

$$p = b_1 2^{-1} + b_2 2^{-2} + \cdots + b_5 2^{-5} + p_r 2^{-5} \quad (7.3.37)$$

where each b_i is 0 or 1, and $0 \leq p_r \leq 1$.

Now imagine that you have generated and stored 64 uniform $U(0, 1)$ deviates, and that the 64-bit word P displays just the first bit of each of the 64. Compare each bit of P to b_1 . If the bits are the same, then we don't yet know whether that uniform deviate is less than or greater than p . But if the bits are *different*, then we know that the generator is less than p (in the case that $b_1 = 1$) or greater than p (in the case that $b_1 = 0$). If we keep a mask of "known" versus "unknown" cases, we can do these comparisons in a bit-parallel manner by bitwise logical operations (see code below to learn how). Now move on to the second bit, b_2 , in the same way. At each stage we change half the remaining unknowns to knowns. After five stages (for $n = 64$) there will be two remaining unknowns, on average, each of which we finish off by generating a new uniform and comparing it to p_r . (This requires a loop through the 64 bits; but since C++ has no bitwise "popcount" operation, we are stuck doing such a loop anyway. If you can do popcounts, you may be better off just doing more stages until the unknowns mask is zero.)

The trick is that the bits used in the five stages are not actually the leading five bits of 64 generators, they are just five independent 64-bit random integers. The number five was chosen because it minimizes $64 \times 2^{-j} + j$, the expected number of deviates needed.

So, the code for binomial deviates ends up with three separate methods: bit-parallel direct, cdf lookup (by bisection), and squeezed ratio-of-uniforms.

```

struct Binomialdev : Ran {
Structure for binomial deviates.
    Doub pp,p,pb,expnp,np,glnp,plog,pclog,sq;
    Int n,swch;
    Ullong uz,uo,unfin,diff,rltp;
    Int pbits[5];
    Doub cdf[64];
    Doub logfact[1024];
    Binomialdev(Int nn, Doub ppp, Ullong i) : Ran(i), pp(ppp), n(nn) {
Constructor arguments are n, p, and a random sequence seed.
        Int j;
        pb = p = (pp <= 0.5 ? pp : 1.0-pp);
        if (n <= 64) {
            uz=0;
            uo=0xffffffffffffffffLL;
            rltp = 0;
            for (j=0;j<5;j++) pbits[j] = 1 & ((Int)(pb * 2.));
            pb -= floor(pb);
            swch = 0;
        } else if (n*p < 30.) {
            cdf[0] = exp(n*log(1-p));
            for (j=1;j<64;j++) cdf[j] = cdf[j-1] + exp(gammln(n+1.)
                -gammln(j+1.)-gammln(n-j+1.)+j*log(p)+(n-j)*log(1.-p));
            swch = 1;

```

deviates.h

Will use bit-parallel direct method.

Leading bits of p (above) and remaining fraction.

Will use precomputed cdf table.

```

    } else {
        np = n*p;
        glnp=gammln(n+1.);
        plog=log(p);
        pclog=log(1.-p);
        sq=sqrt(np*(1.-p));
        if (n < 1024) for (j=0;j<=n;j++) logfact[j] = gammln(j+1.);
        swch = 2;
    }
}

Int dev() {
    Return a binomial deviate.
    Int j,k,kl,km;
    Doub y,u,v,u2,v2,b;
    if (swch == 0) {
        unfin = uo;
        for (j=0;j<5;j++) {
            diff = unfin & (int64()^(pbits[j]? uo : uz)); // Mask of diff.
            if (pbits[j]) rltp |= diff; // Set bits to 1, meaning ran < p.
            else rltp = rltp & ~diff; // Set bits to 0, meaning ran > p.
            unfin = unfin & ~diff; // Update unfinished status.
        }
        k=0;
        for (j=0;j<n;j++) {
            if (unfin & 1) {if (doub() < pb) ++k;} // Now we just count the events.
            else {if (rltp & 1) ++k;} // Clean up unresolved cases,
            unfin >>= 1; // or use bit answer.
            rltp >>= 1;
        }
    } else if (swch == 1) {
        Use stored cdf.
        y = doub();
        kl = -1;
        k = 64;
        while (k-kl>1) {
            km = (kl+k)/2;
            if (y < cdf[km]) k = km;
            else kl = km;
        }
    } else {
        Use ratio-of-uniforms method.
        for (;;) {
            u = 0.645*doub();
            v = -0.63 + 1.25*doub();
            v2 = SQR(v);
            Try squeeze for fast rejection:
            if (v >= 0.) {if (v2 > 6.5*u*(0.645-u)*(u+0.2)) continue;}
            else {if (v2 > 8.4*u*(0.645-u)*(u+0.1)) continue;}
            k = Int(floor(sq*(v/u)+np+0.5));
            if (k < 0) continue;
            u2 = SQR(u);
            Try squeeze for fast acceptance:
            if (v >= 0.) {if (v2 < 12.25*u2*(0.615-u)*(0.92-u)) break;}
            else {if (v2 < 7.84*u2*(0.615-u)*(1.2-u)) break;}
            b = sq*exp(glnp+k*plog+(n-k)*pclog // Only when we must.
                - (n < 1024 ? logfact[k]+logfact[n-k]
                  : gammln(k+1.)+gammln(n-k+1.)));
            if (u2 < b) break;
        }
    }
    if (p != pp) k = n - k;
    return k;
}
};

```

If you are in a situation where you are drawing only one or a few deviates each for many different values of n and/or p , you'll need to restructure the code so that n and p can be changed without creating a new instance of the object and without reinitializing the underlying Ran generator.

7.3.14 When You Need Greater Speed

In particular situations you can cut some corners to gain greater speed. Here are some suggestions.

- All of the algorithms in this section can be speeded up significantly by using `Ranq1` in §7.1 instead of `Ran`. We know of no reason not to do this. You can gain some further speed by coding `Ranq1`'s algorithm inline, thus eliminating the function calls.
- If you are using `Poissondev` or `Binomialdev` with large values of λ or n , then the above codes revert to calling `gammln`, which is slow. You can instead increase the length of the stored tables.
- For Poisson deviates with $\lambda < 20$, you may want to use a stored table of cdfs combined with bisection to find the value of k . The code in `Binomialdev` shows how to do this.
- If your need is for binomial deviates with small n , you can easily modify the code in `Binomialdev` to get multiple deviates ($\sim 64/n$, in fact) from each execution of the bit-parallel code.
- Do you need exact deviates, or would an approximation do? If your distribution of interest can be approximated by a normal distribution, consider substituting `Normaldev`, above, especially if you also code the uniform random generation inline.
- If you sum exactly 12 uniform deviates $U(0, 1)$ and then subtract 6, you get a pretty good approximation of a normal deviate $N(0, 1)$. This is definitely slower than `Normaldev` (not to mention less accurate) on a general-purpose CPU. However, there are reported to be some special-purpose signal processing chips in which all the operations can be done with integer arithmetic and in parallel.

See Gentle [3], Ripley [4], Devroye [6], Bratley [7], and Knuth [8] for many additional algorithms.

CITED REFERENCES AND FURTHER READING:

- Kinderman, A.J. and Monahan, J.F. 1977, "Computer Generation of Random Variables Using the Ratio of Uniform Deviates," *ACM Transactions on Mathematical Software*, vol. 3, pp. 257–260.[1]
- Leva, J.L. 1992. "A Fast Normal Random Number Generator," *ACM Transactions on Mathematical Software*, vol. 18, no. 4, pp. 449–453.[2]
- Gentle, J.E. 2003, *Random Number Generation and Monte Carlo Methods*, 2nd ed. (New York: Springer), Chapters 4–5.[3]
- Ripley, B.D. 1987, *Stochastic Simulation* (New York: Wiley).[4]
- Marsaglia, G. and Tsang W-W. 2000, "A Simple Method for Generating Gamma Variables," *ACM Transactions on Mathematical Software*, vol. 26, no. 3, pp. 363–372.[5]
- Devroye, L. 1986, *Non-Uniform Random Variate Generation* (New York: Springer).[6]

Bratley, P., Fox, B.L., and Schrage, E.L. 1983, *A Guide to Simulation*, 2nd ed. (New York: Springer).[7].

Knuth, D.E. 1997, *Seminumerical Algorithms*, 3rd ed., vol. 2 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), pp. 125ff.[8]

7.4 Multivariate Normal Deviates

A multivariate random deviate of dimension M is a point in M -dimensional space. Its coordinates are a vector, each of whose M components are random — but not, in general, independently so, or identically distributed. The special case of *multivariate normal deviates* is defined by the multidimensional Gaussian density function

$$N(\mathbf{x} | \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{M/2} \det(\boldsymbol{\Sigma})^{1/2}} \exp\left[-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}) \cdot \boldsymbol{\Sigma}^{-1} \cdot (\mathbf{x} - \boldsymbol{\mu})\right] \quad (7.4.1)$$

where the parameter $\boldsymbol{\mu}$ is a vector that is the mean of the distribution, and the parameter $\boldsymbol{\Sigma}$ is a symmetrical, positive-definite matrix that is the distribution's covariance.

There is a quite general way to construct a vector deviate $\mathbf{x}$ with a specified covariance $\boldsymbol{\Sigma}$ and mean $\boldsymbol{\mu}$, starting with a vector $\mathbf{y}$ of independent random deviates of zero mean and unit variance: First, use Cholesky decomposition (§2.9) to factor $\boldsymbol{\Sigma}$ into a left triangular matrix $\mathbf{L}$ times its transpose,

$$\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^T \quad (7.4.2)$$

This is always possible because $\boldsymbol{\Sigma}$ is positive-definite, and you need do it only once for each distinct $\boldsymbol{\Sigma}$ of interest. Next, whenever you want a new deviate $\mathbf{x}$, fill $\mathbf{y}$ with independent deviates of unit variance and then construct

$$\mathbf{x} = \mathbf{L}\mathbf{y} + \boldsymbol{\mu} \quad (7.4.3)$$

The proof is straightforward, with angle brackets denoting expectation values: Since the components y_i are independent with unit variance, we have

$$\langle \mathbf{y} \otimes \mathbf{y} \rangle = \mathbf{1} \quad (7.4.4)$$

where $\mathbf{1}$ is the identity matrix. Then,

$$\begin{aligned} \langle (\mathbf{x} - \boldsymbol{\mu}) \otimes (\mathbf{x} - \boldsymbol{\mu}) \rangle &= \langle (\mathbf{L}\mathbf{y}) \otimes (\mathbf{L}\mathbf{y}) \rangle \\ &= \left\langle \mathbf{L}(\mathbf{y} \otimes \mathbf{y})\mathbf{L}^T \right\rangle = \mathbf{L} \langle \mathbf{y} \otimes \mathbf{y} \rangle \mathbf{L}^T \\ &= \mathbf{L}\mathbf{L}^T = \boldsymbol{\Sigma} \end{aligned} \quad (7.4.5)$$

As general as this procedure is, it is, however, rarely useful for anything except multivariate *normal* deviates. The reason is that while the components of $\mathbf{x}$ indeed have the right mean and covariance structure, their detailed distribution is not anything “nice.” The x_i ’s are linear combinations of the y_i ’s, and, in general, a linear combination of random variables is distributed as a complicated convolution of their individual distributions.